

Module 18: LWC

Lightning Web Components:

In Salesforce, we have 2 types of User Interface.

1. Salesforce Classic --> was the older one.
2. Salesforce Lightning --> is the latest that we are working on.

We have 3 layers in any application:

1. Model --> it is a layer at backend which interacts with Database. (objects, SOQL, SOSL, Apex Triggers)
2. View --> It is a layer at frontend which interacts with the user. (LWC helps in creating the view for salesforce apps)
3. Controller --> It is a layer between the backend and frontend which is used to define the actual logic.(This is done with the help of Apex classes)

To create a view like Salesforce classic, we have a Visualforce Framework. (In visualforce, we have visualforce pages just like our HTML Pages)

To create a view like salesforce Lightning, we have a LWC framework.

--> WE can create a Lightning App. Inside this App, we can have 3 different types of Pages, known as Lightning Pages.(App Page, Record Page, Home Page)

To create Lightning Component in salesforce, we have 2 frameworks:

1. Lightning Aura Component --> It uses Aura Framework, which is third party framework. Aura framework has a Lightning Bundle.(There is a collection of different files --> UI, CSS, Design, JS, Helper)
2. Lightning Web Component --> It uses LWC framework, which is a Salesforce Product. LWC has a Lightning Bundle of 3 files(html, js, XML). It meets all the current web standards.

Agenda

18 Lightning Web Components Basics

18.1 LWC Basics

18.2 Salesforce DX

18.3 DOM Event Handlers

18.4 Base Components

18.5 SLDS

18.6 Applying Custom Styling

18.7 Modern JavaScript Concepts

18.8 Fetching Data Wiring apex with Property

18.1 LWC Basics

Overview of Lightning Web Components

- **Overview of LWC Framework**

- What is LWC?
- Key Features of LWC
- Advantages of LWC

- **Evolution from Aura to LWC**

- Aura Framework
- Transition to LWC
- Key Differences between Aura Framework and LWC

Overview of Lightning Web Components

Lightning Web Components (LWC)

- Modern web standards-based framework for building Lightning components on Salesforce Platform.
- Part of Salesforce's strategy to enhance development experience and performance.

Key Features

- Lightweight and faster rendering due to native support for modern web standards.
- Encourages component-based development, improving reusability and maintainability.

Advantages of LWC

- Seamless integration with other Salesforce technologies.
- Provides a more efficient development cycle with easier debugging and testing

Evolution from Aura to LWC

Aura Framework

- Introduced by Salesforce in 2014 as a robust framework for building Salesforce UI components.
- Based on a proprietary JavaScript framework.

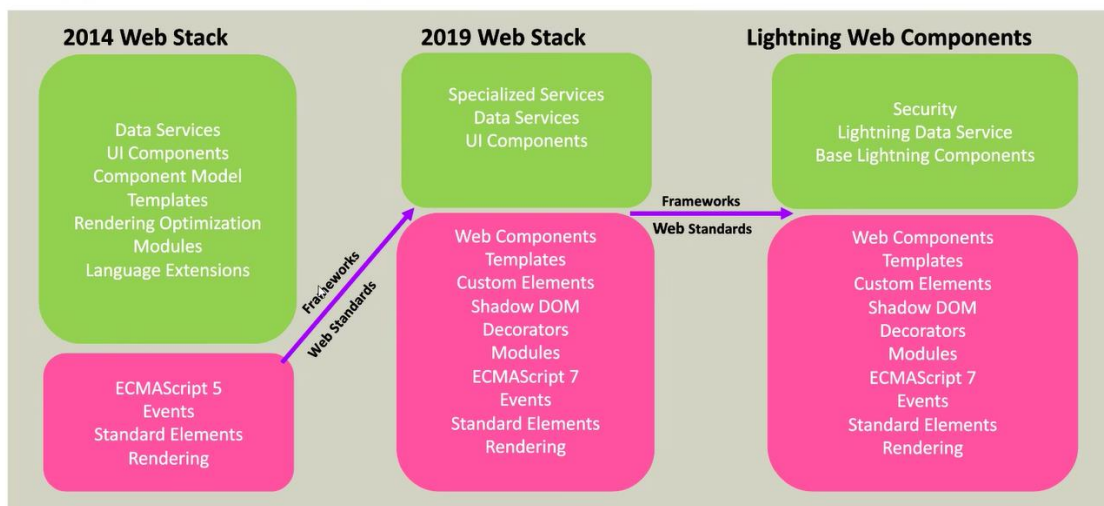
Transition to LWC

- Launched in 2019, LWC represents a shift towards standards-based development.
- Built on top of web components standards (Custom Elements, Shadow DOM, etc.).

Key Differences Between LWC and Aura Framework

- LWC leverages browser's native capabilities for better performance and compatibility.
- Offers a more modern development experience with cleaner syntax and improved flexibility.

Comparison of 2014 and 2019 Approaches



Presentation, Processing, and Storage Logic

- **Presentation Logic**

- HTML markup defines the structure and layout of the component.
- Enhanced with web components encapsulation (Shadow DOM) for scoped styles.

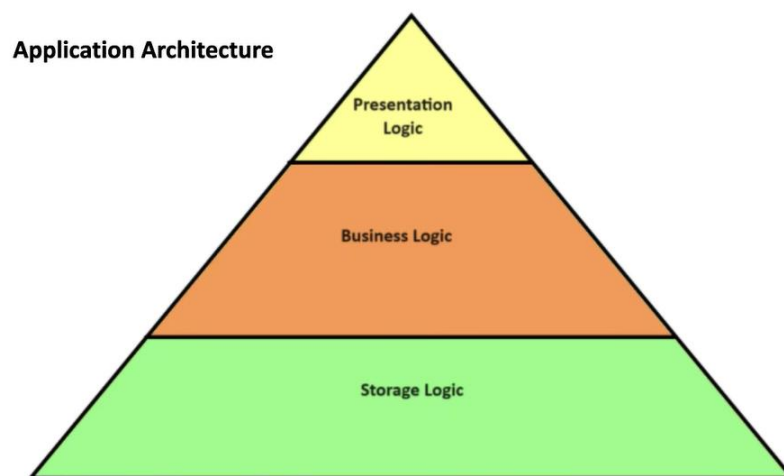
- **Processing Logic**

- Written in JavaScript, manages component behavior and data manipulation.
- Supports modern JavaScript features like async/await for asynchronous operations.

- **Storage Logic**

- Manages state using reactive properties and getters/setters.
- State can be local to the component or shared via properties and events.

Presentation, Processing, and Storage Logic



LWC Bundle Structure

Overview

- Organized structure encapsulating HTML, JavaScript, CSS, SVG, and metadata.
- Facilitates modular development and packaging of components.

Components of a Bundle

Component	Description
HTML Template	Defines component structure and markup
JavaScript Controller	Handles component behavior and interactions
CSS Styling	Scoped styling for the component using standard CSS
SVG	Scalable Vector Graphics used for visual elements
Metadata (XML)	Describes component properties, dependencies, and configuration

18.2 Salesforce DX

Salesforce DX Commands Overview

Create a Project	Salesforce DX projects are containers for all your source and configuration files. They can be created using the <code>force:project:create</code> command. <code>sfdx force:project:create -n MyProjectName</code> This command creates a new Salesforce DX project named <code>MyProjectName</code> in the current directory.
Authorization	Salesforce CLI requires authentication to access Salesforce orgs. You can authorize an org using the <code>force:auth:web:login</code> command. <code>sfdx force:auth:web:login -a MyOrgAlias</code> Initiates a web-based login flow to authorize access to a Salesforce org. <code>-a MyOrgAlias</code> assigns the alias <code>MyOrgAlias</code> to the authenticated org for future reference.

Demo1

Salesforce DX Commands Overview



- 1) Create a Project named LWC_Examples
- 2) Authorize with your Org
- 3) Create a Lightning Web Component named 'helloAccenturite' to display a message 'Hello Accenturite Welcome to LWC'
- 4) Deploy the component to your Org
- 5) Add the component to the Recruiting App Page
- 6) Display the Result

18.3 DOM Event Handlers

Overview of DOM Event Handlers

- DOM events, short for Document Object Model events, are notifications or signals that are generated by elements in a web page when certain actions occur.
- DOM Event Handlers are essential tools in web development that help developers make web pages more dynamic and interactive by responding to user actions using specific functions designed for handling events.

To create LWC, We need Microsoft Visual Studio Code.

You should have VS Code installed, Salesforce CLI installed, JDK installed in your laptops.

Once you have installed VS code, You need to install Salesforce extension pack.

DOM Event Handlers in LWC:

--> Events --> button click, checkbox click, hover, entering any value in textbox, mouseover, mouseout.

--> to handle these kind of events, we have event handlers.

Event handlers are functions that are used to manage these user interactions.

1. Event binding--> binding the event with the event handler method.

```
<button onclick={handleClick}>Click ME </button>
```

2. Event Handler method-->

```
handleClick(){  
  //code  
}
```

3. Event Object--> when the event is triggered, an event object is automatically passed to the handler method.

```
handleClick(event){  
  //code  
}
```

Lightning Base Components:

1. Lightning-card
2. Lightning-button
3. Lightning-input

Demo2

LWC Basics



- 1) Create a below Calculator App using LWC Components, Events and Custom Styling

22 feb:

Today Topics:

1. Decorators in LWC (@track, @wire, @api)
2. Component Lifecycle --> Lifecycle hooks
3. Callback and Arrow Functions
4. Use apex class with LWC(Server side interaction)
 - > @wire with property
 - > @wire with function

Decorators in LWC:

1. @api --> It is used with the property or function defined in js. It is used to define any property as 'public'. It is used to pass values from one component to another component. This we will see in Component Communication Topic.
2. @track --> It is used with the property. It is used to define 'private' property. It makes the property 'reactive'.
3. @wire --> It is used to connect our LWC component with the Apex Class. @wire can be used with the property or function. It is used to get the retrieved data from Apex Class to LWC component. This also makes the property reactive.

LWC Component Lifecycle --> Lifecycle Hooks

Lifecycle Hooks--> These are the methods that allow us to run the code at specific stages in the component Lifecycle.

1. `Constructor()` --> This is the first lifecycle hook called when the component is instantiated. It is used for initialize the values.
2. `ConnectedCallback()` --> This is called when the component is inserted into the DOM.
3. `renderedCallback()` --> This is called after every render of the component.
4. `disconnectedCallback()` --> This is called when the component is removed from the DOM.
5. `errorCallback()` --> used for handling errors at any stage of the component Lifecycle.

18.7 Modern JavaScript

Difference among const, let & var

const	let	var
Declared with block scope, like let, but the value cannot be reassigned after it is initialized. const variables cannot be updated or re-declared.	Declared with block scope, meaning it can only be accessed within the block where it is defined. let variables can be updated but not re-declared.	Can be declared with function or global scope and can be reassigned and updated. var variables are hoisted, meaning they can be accessed before they are declared.

Callback Functions

- A callback function is a function that is passed as an argument to another function.
- It gets executed later, after the first function completes its task.

```
function fetchData(callback) {
  setTimeout(function() {
    const data = 'Fetched data';
    callback(data);
  }, 1000);
}

function processData(data) {
  console.log('Processing data:', data);
}

fetchData(processData);
```

Differences Between Arrow Function & Callback Function

Feature	Callback Functions	Arrow Functions
Definition	Function passed as an argument to another function, execute later.	Concise syntax for writing function expression using '=>'.
Purpose	Handle asynchronous operations, respond to event after task complete.	Write shorter, clearer function definitions, especially for single-expression tasks.
Usage	Passed as an argument to other functions, commonly used in async programming.	Standalone function expression used where function are defined inline or passed directly
Syntax	No specific syntax required being a function.	Use '=>' arrow notation often with implicit return for single expression.

Call back function

- > It is passed as an argument to another function.
- > It is executed later when a certain task is completed.

Arrow Function:

- > It can replace callback function in some scenarios.
- > We use Arrow function because its syntax is easier to use and it is concise.

Arrow Function:

- > It can replace callback function in some scenarios.
- > We use Arrow function because its syntax is easier to use and it is concise.
- > Inside the Arrow function, we can use 'this' keyword easily. Using this keyword we can access the properties directly inside the arrow function.

Considering the above advantages of Arrow Function, we will use this function syntax in our LWC codes.

with sharing --> this means that the apex class will follow the OWD settings, role hierarchy, sharing rules and manual sharing which are configured for Opportunity object.

Opportunity object has 8 records. Out of that 5 records are owned by System admin and 3 records are owned by some user (Andrew).

@AuraEnabled --> This annotation is used inside the apex class with the class method to make that method accessible in lightning component.

To-Do

Fetching Data



Retrieve all records from the Position Object and display them in a table

Position Name	Date Opened	Date Closed	Department	Location
DBA	2012-03-15		IT	Atlanta
Sr. UI Designer	2012-03-20		Engineering	Atlanta
Sr. Director, Data Center	2012-04-07	2012-07-01	IT	San Francisco
SW Engineer	2012-04-07		Engineering	San Francisco
Sr. SQA Engineer	2012-05-15	2012-06-20	Engineering	San Francisco
Documentation Writer	2012-01-06		Engineering	San Francisco
Help Desk Analyst	2012-06-15		IT	New York
Network Technician	2012-05-15		IT	San Francisco
Oracle Developer	2012-06-18		IT	New York
Sr Salesforce Developer	2012-07-30	2012-09-27	IT	San Francisco
UI Designer	2012-12-04		Engineering	Atlanta

To-Do

Fetching Data



Retrieve all open positions records in tabular format, with the location column values visible in upper case.

Position Name	Date Opened	Date Closed	Department	Location
DBA	2012-03-15		IT	ATLANTA
Sr. UI Designer	2012-03-20		Engineering	ATLANTA
SW Engineer	2012-04-07		Engineering	SAN FRANCISCO
Documentation Writer	2012-01-06		Engineering	SAN FRANCISCO
Help Desk Analyst	2012-06-15		IT	NEW YORK
Network Technician	2012-05-15		IT	SAN FRANCISCO
Oracle Developer	2012-06-18		IT	NEW YORK
UI Designer	2012-12-04		Engineering	ATLANTA

-->Imperative Approach of calling Apex methods

-->Lightning Data Table

-->ShowToastEvent

-->component Communication:

We have 2 approach to call Apex class Methods:

1. Using @wire annotation

--> Using @wire with a property

--> Using @wire with a function

2. Imperative Approach --> Based on the user interaction, the method is going to be called.

19.1 Fetching Data

JavaScript Promise

Definition of a Promise

- An object representing the eventual completion or failure of an asynchronous operation.
- Provides a clean solution to handle callbacks and manage asynchronous code.

Key Components

States: Pending, Fulfilled, Rejected

- **Pending:** Initial state of a promise.
- **Fulfilled:** Promise successfully resolved.
- **Rejected:** Promise failed to resolve.

19.1 Fetching Data

JavaScript Promise Example

```
// Using the addAsync function with promises
addAsync(5, 3)
  .then(result => {
    console.log('Sum:', result); // Output: Sum: 8
  })
  .catch(error => {
    console.error('Error:', error.message);
  });

addAsync('two', 3)
  .then(result => {
    console.log('Sum:', result);
  })
  .catch(error => {
    console.error('Error:', error.message); // Output: Error: A
  });
```

LWC Approach

```
function addAsync(a, b) {
  return new Promise((resolve, reject) => {
    // Simulating some asynchronous operation with setTimeout
    setTimeout(() => {
      if (typeof a !== 'number' || typeof b !== 'number') {
        reject(new Error('Arguments must be numbers'));
      } else {
        resolve(a + b);
      }
    }, 1000); // Simulating a delay of 1 second
  });
}
```

JavaScript behind the scene

Imperative Method Call

Imperatively calling an Apex method in Lightning Web Components is particularly useful in scenarios where you need to fetch data on-demand or perform data manipulation operations (DML) such as inserts, updates, or deletions.

Fetching Data On-Demand:

- **User Interaction:** You may want to fetch data from Salesforce only when a user clicks a button or interacts with a particular part of your component.
- **Dynamic Parameters:** Imperative calls allow you to pass parameters dynamically, which is useful when the parameters are based on user input or other runtime conditions.
- **Control Flow:** Unlike `@wire`, imperative calls give you more control over when and how often the Apex method is invoked, which can be crucial for performance optimization and UX design.

Imperative Method Call

Performing DML Operations:

- **Form Submissions:** When a user submits a form and you need to insert or update records in Salesforce, imperative Apex calls are necessary.
- **Validation and Logic:** Sometimes, client-side logic in your LWC needs to coordinate with server-side logic (Apex) to validate data or perform complex operations that are not feasible or secure to do entirely on the client side.

Wiring Apex With A Function

Step 4: HTML code for opportunityCompV3.html

```
<template>
<lightning-card title="Opportunity Using imperative Approach" icon-name="standard:opportunity">
  <div slot="actions">
    <lightning-button variant="success" label="Won Deals" onclick={loadWonDeals}></lightning-button>
    <lightning-button variant="destructive" label="Lost Deals" onclick={loadLostDeals}></lightning-button>
  </div>
  <template lwc:if={opplist}>
    <table class="slds-table slds-table_bordered slds-table_col-bordered slds-table_stripped">
      <thead>
        <tr>
          <th>Name</th>
          <th>Stage Name</th>
          <th>Close Date</th>
          <th>Amount</th>
        </tr>
      </thead>
      <tbody>
        <template for:each={opplist} for:item="opp">
          <tr key={opp.id}>
            <td>{opp.Name}</td>
            <td>{opp.StageName}</td>
            <td>{opp.CloseDate}</td>
            <td>{opp.Amount}</td>
          </tr>
        </template>
      </tbody>
    </table>
  </template>
</lightning-card>
</template>
```

To-Do

Fetching Data



Create three buttons on the lightning card labeled "New," "Open," and "Close" to load the position records in tabular format when the buttons are clicked.

Positions Using Imperative Approach						New Position	Open Position	Close Position
Position Name	Date Opened	Date Closed	Department	Location	Status			
DBA	2012-03-15		IT	Atlanta	Open			
Sr. UI Designer	2012-03-20		Engineering	Atlanta	Open			
SW Engineer	2012-04-07		Engineering	San Francisco	Open			
Documentation Writer	2012-01-06		Engineering	San Francisco	Open			
Help Desk Analyst	2012-06-15		IT	New York	Open			
Network Technician	2012-05-15		IT	San Francisco	Open			
Oracle Developer	2012-06-18		IT	New York	Open			
UI Designer	2012-12-04		Engineering	Atlanta	Open			

Demo 8

lightning-datatable



- Using the Lightning database table, retrieve all opportunity records in tabular style.
- Define delete button functionality to delete a specific record from LDT using Apex and UI Record API However closed won deal can not delete.
- Display a success and error message in toast.
- Create personalized labels for success and failure in English and Spanish.

Demo 8

lightning-datatable



- Using the Lightning database table, retrieve all opportunity records in tabular style.
- Define delete button functionality to delete a specific record from LDT using Apex and UI Record API However closed won deal can not delete.
- Display a success and error message in toast.
- Create personalized labels for success and failure in English and Spanish.

					Delete
	Name	Stage Name	Closed Date	Amount	
1	☒ Edge Emergency Generator	Closed Won	29-May-2024	\$75,000.00	
2	☐ University of AZ Portable Generators	Closed Won	28-Feb-2024	\$50,000.00	
3	☐ Express Logistics Standby Generator	Closed Won	24-Feb-2024	\$2,20,000.00	
4	☐ GenePoint Standby Generator	Closed Won	02-Apr-2024	\$85,000.00	

Topic: Component Communication

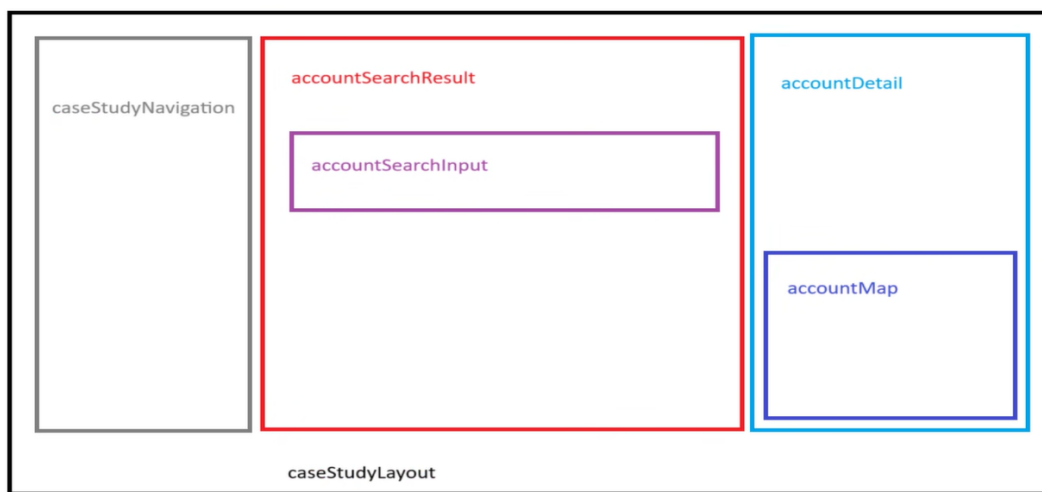
Today's topic:

1. Component Communication : To pass the data from one LWC component to another |

- > Communication between related components
 - > Parent to child component communication
 - > Child to Parent component communication
- > Communication between unrelated components
 - > Lightning Message service(LMS)--> Publish subscribe model

Case Study Layout

LWC



Today's topic:

1. Component Communication : To pass the data from one LWC component to another LWC component is known as Component Communication.

This is of 2 types:

1. Communication between related components: If we call one component inside another component, it becomes the parent-child relationship between the components. The outer one is Parent comp and the inner one is child comp.

- > Parent to child component communication
- > Child to Parent component communication

2. Communication between unrelated components

- > Lightning Message service(LMS)--> Publish subscribe model

data from parent(outer) comp to child(inner) comp is 'Parent to child' communication.

This is implemented with the help of @api decorator.

--> Child to Parent component communication: Means passing the data from child(inner) comp to parent(outer) comp is 'Child to Parent' communication.

I

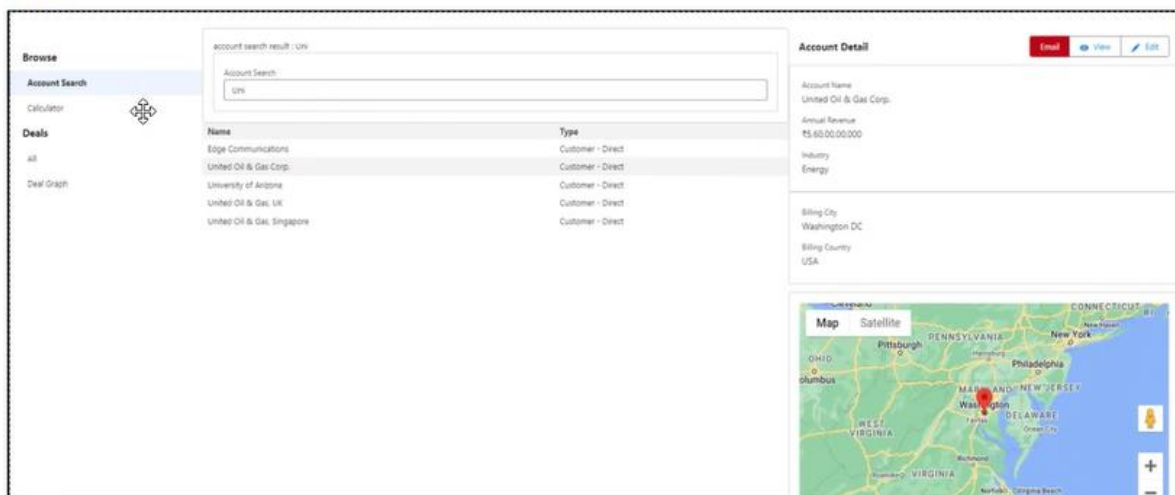
This is implemented with the help of 'Custom Events'.

2. Communication between unrelated components

--> Lightning Message service(LMS)--> Publish subscribe model

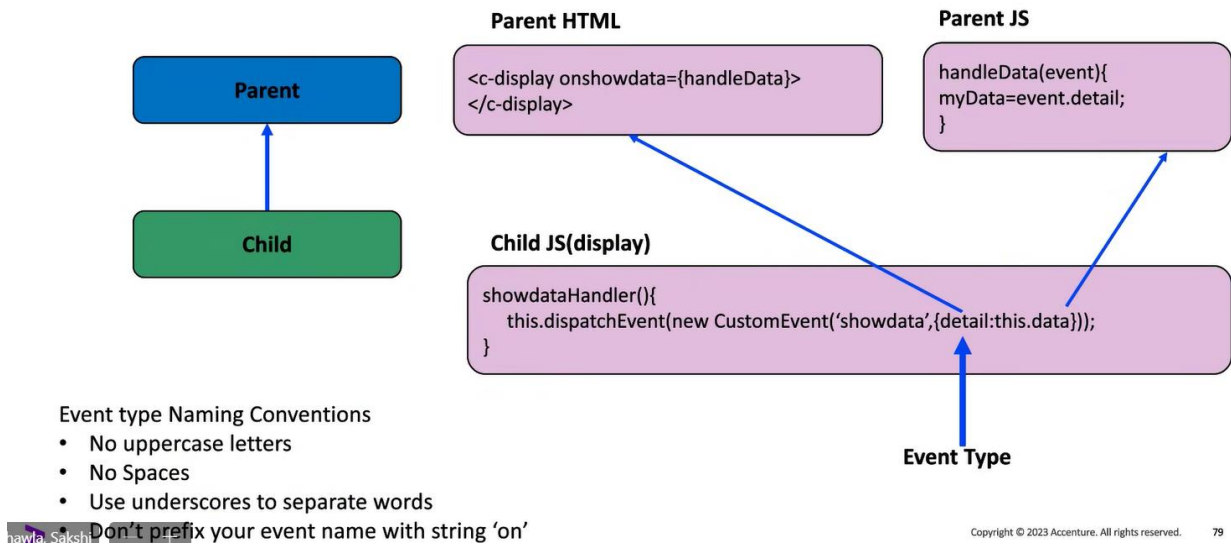
Mini Case Study

LWC



19.4 Component Communication

Child-Parent Communication



Copyright © 2023 Accenture. All rights reserved. 79

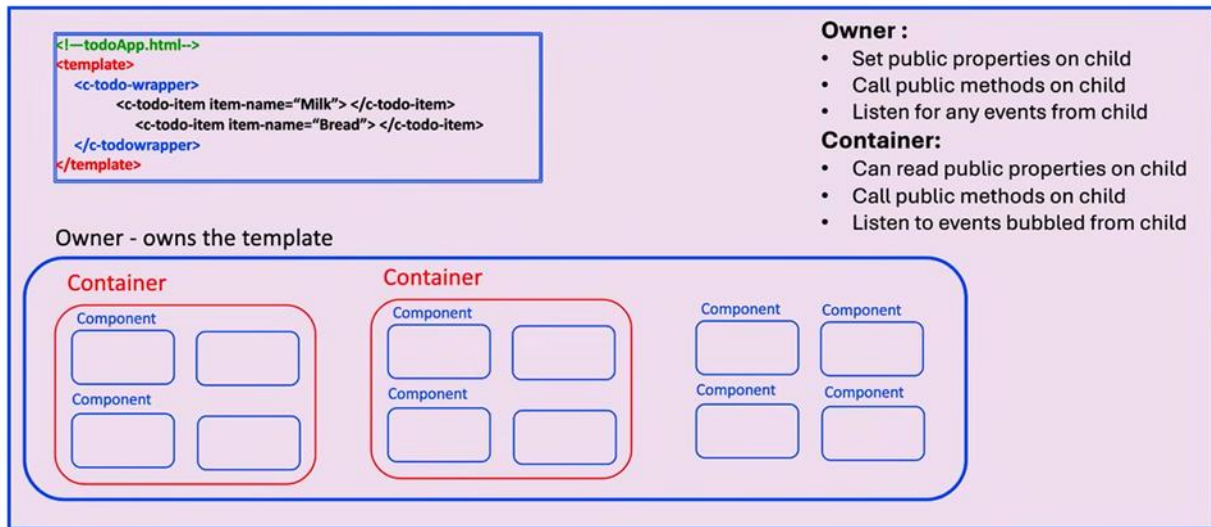
Demo10

Component Communication

- Send data from child component to a parent component as well as grand parent component.



Overview of Component Communication



2. Communication between unrelated components - means if we want to pass the data from one component to another and both of these components are separate(stand-alone) and no component is part of another component.

--> Lightning Message service(LMS)--> Publish subscribe model

1. One component is treated as 'Publisher' component.

2. Other component is treated as 'Subscriber' component.

To communicate between publisher and subscriber component, we need to have a 'Message Channel'.

28 Jan

SLDS --> Salesforce Lightning Design System.

LMS --> Lightning Message Service(Component communication between unrelated components)

Lightning Data Service(LDS): This service helps us to interact with the database without the need of Apex class. we can perform basic CRUD operations with LDS without the need of Apex class.

--> If we want to fetch a single record from the database and display its details on LWC, then we can use LDS. Likewise, we can create, update and delete a single record by providing the record id.

Lightning Message Service

Overview of Lightning Message Service

